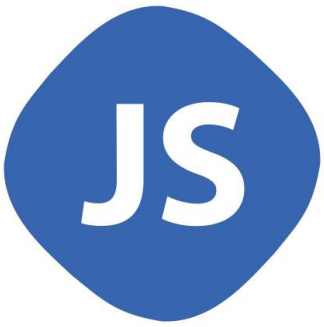


Простые циклы

- Введение
- Цикл for
- Цикл while
- Цикл do ... while
- Сохранение коллекций
- Бесконечные циклы
- Инструкции break и continue
- Метка (label)

Введение

Каждый раз, когда мы занимались выводом данных в браузер, мы делали это вручную. Таким образом, большой объем данных не выгрузишь (и не загрузишь;). Пришло время автоматизировать процесс.

Цель изучения циклов – приобрести навык итерирования следующих структур:

- **HTML–коллекций любого уровня вложений**
- **Массивов / вложенных массивов**
- **Объектов / сложных объектов**
- **Особых коллекций Map и Set**

Циклы JavaScript обеспечивают многократное выполнение повторяющихся вычислений. Они оптимизируют процесс написания кода, выполняя одну и ту же инструкцию или блок инструкций, образующих **тело цикла**, заданное число раз (используя переменную-счётчик) или пока заданное условие истинно.

Циклы выполняют обход последовательности некоторого набора значений.

> ***К сведению***. Однократное выполнение цикла называется **итерацией**.

На производительность цикла влияют количество **итераций** и количество **операций**, выполняемых в теле цикла каждой итерации.

В JavaScript существуют следующие операторы цикла:

- **for** используется, когда вы заранее знаете, сколько раз вам нужно что-то сделать;
- **while** используется, когда вы не знаете, сколько раз нужно что-то сделать;
- **do ... while** работает аналогично с оператором **while**. Отличается тем, что do...while всегда выполняет выражение в фигурных скобках, по крайней мере один раз, даже если проверка условия возвращает **false**.
- **for ... of** используется для обхода **итерируемых объектов** (строк, коллекций, массивов и некоторых других);
- **for ... in** используется для обхода **свойств объектов**.

Цикл for

Самый сложный, но при этом самый распространенный цикл - цикл **for**.

Данный цикл в основном используется, когда известно **точное количество повторений** (**итераций**). Этот цикл ещё называют циклом со **счётчиком**.

Синтаксис

```
for ([initial-expression]; [condition]; [final-expression]) {
    statement;
};
```

Параметры

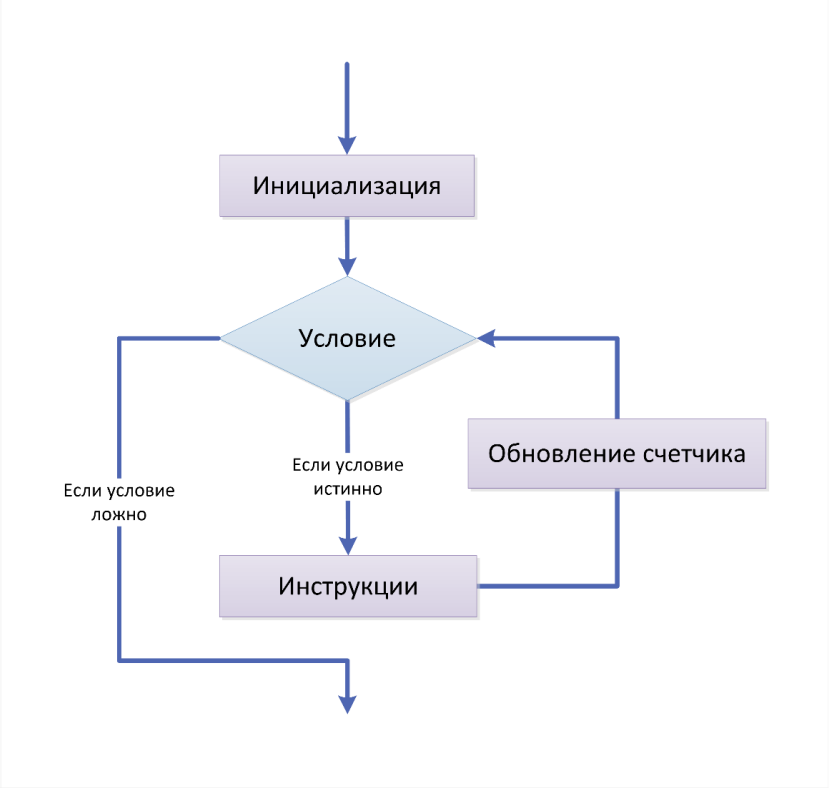
- **initial-expression** (инициализация) - выражение, которое выполняется **один раз** перед выполнением цикла. Обычно используется для инициализации счётчика;
- **condition** (условие) - выражение, истинность которого проверяется **перед каждой итерацией**. Если выражение вычисляется как истина, то

выполняется итерация, в противном случае цикл **for завершает** работу;

- **final-expression** (финальное выражение) - выражение, которое выполняется в конце каждой итерации. Обычно используется для изменения счетчика.
- **statement** (тело цикла) - одна или несколько инструкций, выполнение которых нужно повторять. При этом если тело цикла состоит из одной инструкции, то её **можно не заключать в фигурные скобки**.

Ни один из параметров не является обязательным.

Схематически цикл можно представить следующим рисунком:



Следующий цикл выведет на консоль строчку "Hello, JavaScript!" пять раз, совершит пять итераций.

example_1. Тестируем цикл for

```
<script>

    // цикл for
```

```
    for (var i = 0; i < 5; i++) {
        console.log(i + ": Hello, JavaScript!");
    }
</script>
```

Как работает цикл for:

- **инициализация**: `var i = 0` (объявление переменной i и присвоение ей значения 1);
- **условие** выполнения цикла: `i < 5` (пока значение переменной i меньше 5);
- **тело цикла**: одна или несколько инструкций, которые нужно выполнять: `console.log (i + ": Hello, JavaScript!")`.
- **финальное выражение**, которое нужно выполнять в конце каждой итерации: `i++` (увеличение значение переменной i на 1);

Необязательные части цикла цикла for

Ни одна часть цикла for не является обязательной.

Например, можно пропустить выражение **инициализации**. В этом случае инициализацию переменной можно вынести за пределы цикла.

```
var i = 0;
// цикл for
for (; i < 5; i++) {
    console.log(i + ": Hello, JavaScript!");
}
```

Условие тоже не является обязательным. Без условия **цикл будет выполняться бесконечное количество раз**. В этом случае чтобы его прервать (выйти из цикла) необходимо использовать инструкцию **break**.

```
// цикл for
for (var i = 0; ; i++) {
```

```
        if (i == 5) { // условие прерывания цикла
            break;
        }
        console.log(i + ": Hello, JavaScript!");
}
```

Финальное выражение в for также является не обязательным. Счётчик цикла в этом случае можно, например, изменять в теле.

```
// цикл for
for (var i = 0; i < 5; ) {
        console.log(i + ": Hello, JavaScript!");
        i++; // увеличение счетчика на 1
}
```

Таким образом, в общем случае, цикл **for** можно записать в виде выражения бесконечного цикла.

example_2. Цикл for без параметров

```
<script>
    var i = 0;
    // цикл for
    for (;;) {
        // условие прерывания цикла
        if (i == 5) break;
        console.log(i + ": Hello, JavaScript!");
        i++; // увеличение счетчика на 1
    }
</script>
```

Перебор элементов коллекции циклом for



Цикл **for** используется для выполнения итераций по элементам **коллекций**, **массивов** или **объектов**. Цикл **for** повторяет действия, пока не произойдёт какое-либо специальное событие завершения цикла.

Чтобы вывести элементы коллекции с помощью цикла **for**, нужно задействовать свойство коллекции **length**.

> ***К сведению***. С помощью свойства **length** определяется количество элементов в коллекции и, соответственно, **количество итераций** цикла.

Цикл должен выполниться такое же **количество раз**.

Приведённый ниже скрипт создает коллекцию элементов **p** с классом sel и выводит каждый элемент коллекции на экран в виде **заголовков четвертого уровня**.

example_3. Перебор элементов коллекции циклом for

```
<script>
    document.write("<h2>Динамический вывод элементов коллекции</h2>");
    // получение коллекции элементов p с классом sel
    let elements = document.querySelectorAll('#citate p.sel');
    // цикл for по элементам коллекции
    for (var i = 0; i < elements.length; i++) {
        // доступ к элементу коллекции через индекс
        document.write("<h4>", elements[i].innerText, "</h4>");
    }
</script>
```

> ***Важно***. **Узел** DOM, **доступ** к узлу DOM, **свойства** узлов, – без знания этих понятий невозможно организовать **правильный перебор** элементов коллекции. Именно для этого мы потратили на узлы целую тему.

Если значение свойства **length** не изменяется в ходе выполнения цикла, можно сохранить его в **локальной переменной**, а затем использовать эту переменную в условном выражении. Таким образом, можно повысить скорость выполнения цикла, так как значение свойства **length** будет извлекаться всего **один раз** за всё время работы цикла.

```
let len = elements.length;

// цикл for

for (var i = 1; i <= len; i++) { ... }
```

Вот теперь **объем** выводимых данных перестает существовать. Циклы будут делать все за нас. Ну, почти все...

Цикл while

Цикл while предназначен для многократного выполнения одних и тех же инструкций до тех пор, пока истинно некоторое условие. Цикл в основном используется, когда количество повторений заранее **не известно**.

После завершения цикла программа продолжит работу с первой строки, следующей непосредственно после цикла (если таковая имеется).

Синтаксис

```
while (condition) {
      statement;
};
```

Параметры

- **condition** - условие цикла.
- **statement** - блок инструкций для выполнения во время итераций. Будет выполняться, пока верно условие condition.

Цикл **while** - цикл с **предварительной** проверкой условного выражения. Истинность условия проверяется перед каждым выполнением. Если перед первой итерацией условие ложно, то цикл не выполнится ни разу.

Схематически цикл можно представить следующим рисунком:



Предложенный скрипт выведет на экран таблицу умножения для числа 3.

example_4. Тестируем цикл while

```
<script>
    // инициализируем переменные цикла
    var i = 1;
    var msg = '';
    // цикл while
    while (i <= 10) {
        // формируем строку
        msg += i + ' x 3 = ' + (i * 3) + '\n';
```

```
        i++;
    }
    // выводим строку в консоль
    console.log(msg);
</script>
```

Перебор элементов коллекции циклом while

Следующий скрипт получает коллекцию строк таблицы (HTML-узлы tr), в цикле создает коллекцию ячеек **td** текущей строки и выводит в виде **нумерованного списка**:

- название
- дату выпуска
- статус альбома

example_5. Перебор элементов коллекции циклом while

```
<script>
    // получение коллекции строк таблицы
    var elements = document.querySelectorAll('tbody tr');
    // количество элементов коллекции
    let len = elements.length;
    // инициализируем переменные цикла
    var line = '';
    var i = 0;
    // цикл while
    while (i < len) {
        // получаем коллекцию элементов td для каждой строки tr
        let td = elements.item(i).children;
        // формируем строку для вывода в браузер
```

```
            line += "<li>" + td.item(1).innerText;
            line += "(" + td.item(2).innerText + " / ";
            line += td.item(5).innerText + ")</li>";
            i++;
        }
        let html = "<h2>Динамический список дискографии группы Pink Floyd</h2>";
        html += "<ol>" + line + "</ol>";
        // выводим сформированный контент в браузер
        document.write(html);
</script>
```

Цикл do ... while

Цикл **do...while**, также как и цикл **while**, выполняет одни и те же инструкции до тех пор, пока указанное условие **истинно**. Но в отличие от while в **do...while** условие проверяется **после выполнения инструкций**. Поэтому, цикл **do...while** в любом случае выполнится как минимум **один раз**, даже если условие изначально ложно.

Синтаксис

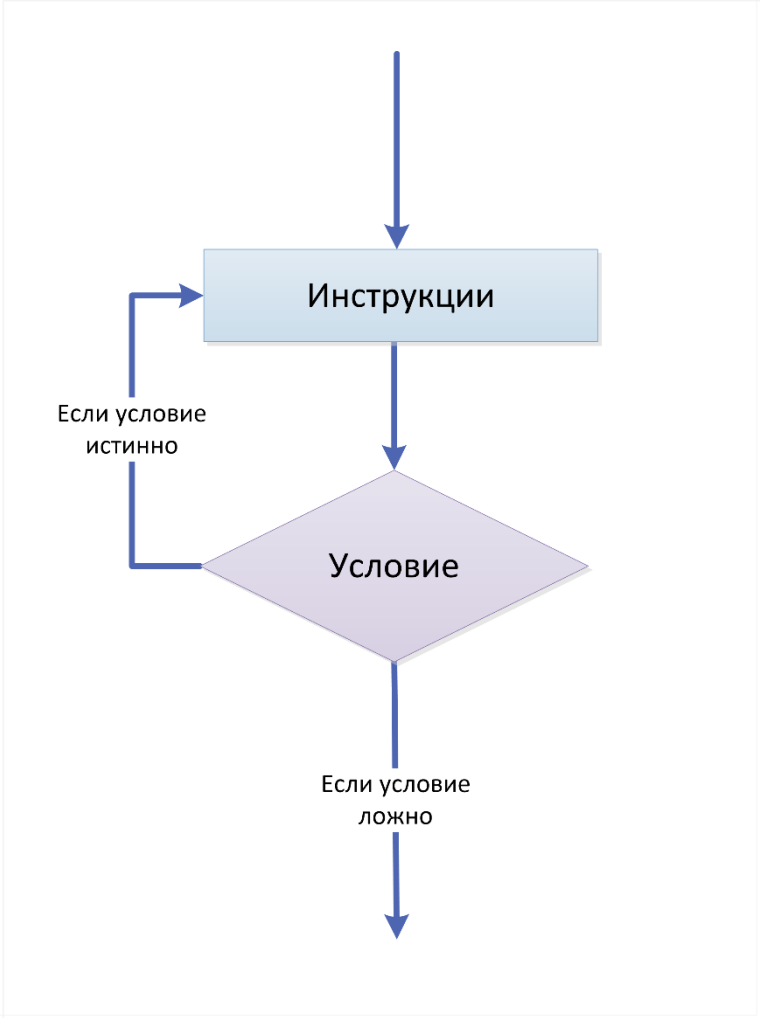
```
do {
     statement;
} while (condition)
```

Параметры

- **statement** - блок инструкций, который будет выполняться пока верно условие condition.
- **condition** - условие цикла. Будет вычисляться на каждой итерации, если оно верно, **statement** будет выполнено, если нет - управление передается на операторы после цикла.

Еще раз. Цикл **do...while** всегда выполнится **хотя бы один раз**.

Схематически цикл можно представить следующим рисунком:



Данный цикл используется реже, чем **while**, так как на практике ситуация, когда требуется хотя бы однократное исполнение цикла, встречается редко.

В следующем примере операторы внутри цикла выполняются один раз, не смотря на то, что условие не выполняется.

example_6. Тестируем цикл do ... while

```
<script>

    var i = 11;

    // цикл do
```

```
    do {
        // выполняем инструкцию цикла
        document.write(i + ": Hello, JavaScript!");
        i++;
    } while (i < 10); // проверяем условие
</script>
```

Перебор элементов коллекции циклом do ... while

Следующий скрипт получает коллекцию элементов **li** нумерованного списка, далее в цикле получает элементы **li вложенного списка** и выводит сформированную строку в виде **маркированного списка**.

example_7. Перебор элементов коллекции циклом do ... while

```
<script>
    // получаем коллекцию элементов li
    var elements = document.getElementsByTagName('ol')[0].children;
    // количество элементов коллекции
    let len = elements.length;
    // инициализация переменных цикла
    var line = "";
    var i = 0;
    // цикл do
    do {
        // получаем текстовый узел - название фильма
        let el = elements[i].firstChild;
        // получаем элементы вложенного списка li
        let li = elements[i].getElementsByTagName("li");
        // формируем элементы списка для вывода
```

```
        line += `
            <li>${el.data} (
            ${li[2].innerText} /
            ${li[0].innerText} /
            ${li[1].innerText}
            )</li>
        `;
        i++;
    } while (i < len);
    line = "<ul>" + line + "</ul>";
    // добавляем заголовок
    let html = "<h2>Динамический список лучших фильмов</h2>" + line;
    // выводим список в браузер
    document.write(html);
</script>
```

Примечание. Получить доступ к коллекции и вывести ее здесь же в браузер или консоль, какой в этом смысл? Пока никакого. А вот **перебрать коллекцию** и сохранить ее в **массиве** или **объекте** - смысл большой. Следующим шагом возможен обмен сохраненных данных с другим сценарием или передача базе данных.

Сохранение коллекций

Переработаем код сценария **example_3**. На этот раз сохраним коллекцию элементов в **одномерный массив**.

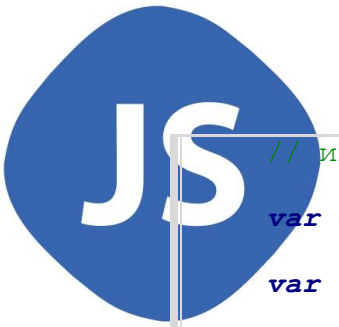
example_8. Сохранение данных в одномерный массив

```
<script>
    // объявляем массив
    var arr = [];
    // получаем коллекцию элементов p
    let elements = document.querySelectorAll('#citate p.sel');
    // циклом for заполняем массив
    for (var i = 0; i < elements.length; i++) {
        arr[i] = elements[i].innerText;
    }
    // упакуем массив в JSON формат
    arrJSON = JSON.stringify(arr);
    document.write("<h2>Упакованный в JSON массив</h2>");
    // массив готов к передаче (например по сети)
    // вывод в браузер
    document.write(arrJSON);
    // вывод в консоль
    console.log(arr);
</script>
```

Переработаем код сценария **example_9**. Сохраним коллекцию элементов в **массив**, состоящий из **объектов** (массив объектов).

example_9. Сохранение данных в массив объектов

```
<script>
    // получаем коллекцию строк таблицы
    var elements = document.querySelectorAll('tbody tr');
    // количество элементов коллекции
    let len = elements.length;
```



```
    // инициируем переменные
    var arr = [];
    var i = 0;
    // цикл while
    while (i < len) {
        // получаем коллекцию элементов td для каждой строки tr
        let td = elements.item(i).children;
        // объявляем пустой объект
        obj = {};
        // свойствам объекта присваиваем значения ячеек
        obj.name =  td.item(1).innerText;
        obj.date = td.item(2).innerText;
        obj.status = td.item(5).innerText;
        // записываем объект в массив
        arr[i] = obj;
        // увеличиваем итератор цикла
        i++;
    }
    // упакуем массив объектов в JSON-формат
    var arrJSON = JSON.stringify(arr);
    document.write("<h2>Упакованный в JSON массив</h2>");
    // выводим сформированный JSON-контент в браузер
    document.write(arrJSON);
    // в консоль
    console.log(arr);
</script>
```



Мы рассмотрели основные операторы циклов языка JavaScript. Не бывает хороших или плохих операторов циклов. Каждым циклом оптимально перебирать определенные типы структур.

Специальные операторы циклов **for ... of** и **for ... in**, проходящие по перечислимым объектам и свойствам объектов рассмотрим в следующем уроке.

Бесконечные циклы

При создании любого цикла можно получить ситуацию **бесконечного цикла**, который никогда не завершится. Такой цикл может потенциально продолжать работать до тех пор, пока работает компьютер пользователя.

Большинство современных браузеров могут обнаружить это и предложат пользователю остановить выполнение скрипта. Чтобы избежать создания бесконечного цикла, вы должны быть уверены, что **заданное условие в какой-то момент вернёт false**.

Например, следующий цикл задаёт условие, которое никогда не возвращает **false**, так как переменная **i** никогда не будет меньше **10**:

example_10. Тайна вечного двигателя

```
<script>
    for (var i = 15; i > 10; i++) {
        document.write("Эта музыка будет вечной...<br>");
    }
</script>
```

Инструкции break и continue

Внутри тела цикла можно использовать специальные инструкции: **break** и **continue**.



Инструкция **break** предназначена для **прекращения выполнения текущего цикла**. Другими словами, она осуществляет выход и передачу управления инструкции, идущей после этого цикла.

Ниже представлен пример кода, в котором **прерывается** цикл по перебору элементов массива, если его текущий элемент равен 5:

example_11. Инструкция страдающим бессонницей

```
<script>
    var i = 1;
    document.write("<h2>Считаем слоников</h2>");
    while (i <= 10) {
        document.write (i + "-й слоник <p>");
        // прекращаем цикл при i == 7
        if (i == 7) break;
        i++;
    }
    document.write("<h3>Первая инструкция после цикла</h3>");
</script>
```

Инструкция **continue** предназначена для прекращения дальнейшего выполнения кода и перехода к **следующей итерации цикла**.

Пример кода, в котором при переменной цикла i=5 прерывается текущая итерация цикла.

example_12. В поисках динозаврика

```
<script>
    var i = 0;
    document.write("<h2>Считаем динозавриков</h2>");
    do {
        i++;
```

```
        // прерываем цикл при i == 5
        if (i == 5) continue;
        document.write (i + "-й динозаврик<br>");
    } while (i < 9);
    document.write("<hr>");
    document.write("5-й динозаврик пропал :(");
    document.write("<h3>Вы 5-го динозаврика не видели??</h3>");
</script>
```

Метка (label)

Метка представляет собой оператор с идентификатором, который позволяет вам **ссылаться на какое-то место в вашей программе**. Например, вы можете использовать метку, чтобы обозначить **любой** из вложенных циклов, а затем использовать операторы **break** или **continue**, чтобы указать, должна ли программа прерывать **указанный** цикл при выполнении определенных условий.

Синтаксис

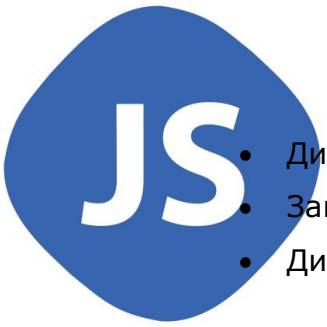
```
Имя_метки:
    оператор;
```

Значение метки может быть любым корректным JavaScript идентификатором, не являющимся зарезервированным словом. Оператор, указанный после метки может быть любым выражением.

Подробнее метки будут рассмотрены в уроке посвященном вложенным циклам.

В **самостоятельных работах** к уроку рассмотрим следующие вопросы:

- Создание коллекций элементов из таблиц и вложенных списков.



- Динамическое построение таблиц.
- Заполнение массива в цикле.
- Динамическое построение выпадающих списков.

